SentinAir

Automated Air Quality Classification System

MLOps Final Project Report

Course: DA5402 — Machine Learning Operations

Name: Singara Harendra

Roll Number: DA25M028

Dataset	UCI Air Quality Dataset (9,358 rows × 13 sensors)
Models	Random Forest, Logistic Regression, SVM
Stack	Airflow · MLflow · FastAPI · Streamlit · Prometheus · Grafana
Platform	Docker Compose (LocalExecutor + PostgreSQL)

Submitted: April 2026

1 Introduction

SentinAir is an end-to-end MLOps application that classifies urban air quality into three categories — *Good*, *Moderate*, and *Poor* — using multivariate chemical sensor data derived from the UCI Air Quality dataset. The project demonstrates a complete production-grade pipeline covering data ingestion, preprocessing, model training, experiment tracking, REST inference, interactive frontend, real-time monitoring, and automated feedback-driven retraining.

The system is designed around five core MLOps principles:

1. **Reproducibility** — DVC tracks raw data versions; MLflow tracks every experiment run.
2. **Automation** — Apache Airflow schedules daily training and hourly retraining pipelines.
3. **Observability** — Prometheus scrapes live metrics; Grafana renders near-real-time dashboards.
4. **Loose Coupling** — Frontend communicates with backend exclusively via REST API.
5. **Continuous Improvement** — A feedback loop collects user corrections and triggers conditional retraining based on error rate and data drift.

2 System Architecture

2.1 High-Level Architecture

SentinAir is organized into six layers that interact through well-defined interfaces:

Layer	Technology	Responsibility
Data Layer	DVC, CSV files	Raw & intermediate data versioning
Orchestration Layer	Apache Airflow 2.9.3	DAG scheduling, task execution
Experiment Tracking	MLflow	Run logging, model registry
Inference Layer	FastAPI + Uvicorn	REST prediction & feedback API
Frontend Layer	Streamlit	User-facing prediction UI
Monitoring Layer	Prometheus + Grafana	NRT metrics & alerting

Table 1: SentinAir layer breakdown

2.2 Technology Stack

Component	Technology	Port
Orchestration	Apache Airflow 2.9.3	8080
Experiment Tracking	MLflow	5000
Inference API	FastAPI + Uvicorn	8000
Frontend	Streamlit	8501
Database	PostgreSQL 13	5432
Monitoring	Prometheus	9090
Dashboards	Grafana	3000
Alerting	Alertmanager	9093
Containerisation	Docker + Docker Compose	—
Data Versioning	DVC	—
ML Framework	scikit-learn	—

Table 2: Full technology stack with service ports

2.3 Data Flow

The end-to-end data flow is as follows:

1. Raw CSV is ingested and written to `data/processed/intermediate/full_data.csv`.
2. Preprocessing splits the dataset (80/20 chronological), applies feature selection, label encoding (Good → 0, Moderate → 1, Poor → 2), and fits a scikit-learn `Pipeline` (median imputation + StandardScaler), saving it as `fitted_preprocessor.pkl`.
3. Model training evaluates Random Forest, Logistic Regression, and SVM; the best model is saved as `best_model.pkl` and all runs are logged to MLflow.
4. FastAPI loads `best_model.pkl` at startup and serves predictions at `/predict`.
5. Streamlit calls the REST API and displays results to users.
6. User feedback (corrected labels) is stored in the PostgreSQL `feedback` table.
7. The hourly retraining DAG checks error rate and data drift, then conditionally merges feedback into the training set and re-trains.

3 Apache Airflow Pipelines

3.1 DAG 1: air_quality_dag (Daily Training Pipeline)

The primary training DAG runs daily and executes three sequential tasks:

1. **data_ingestion** — Loads `data/raw/data.csv`, validates the schema, and writes to the intermediate directory.
2. **preprocessing** — Performs train/test split, feature selection, label encoding (with the training mapping reused for the test set to prevent data leakage), and preprocessor fitting.

3. **model_training** — Trains all three classifiers, logs metrics and artifacts to MLflow, and saves the best model.

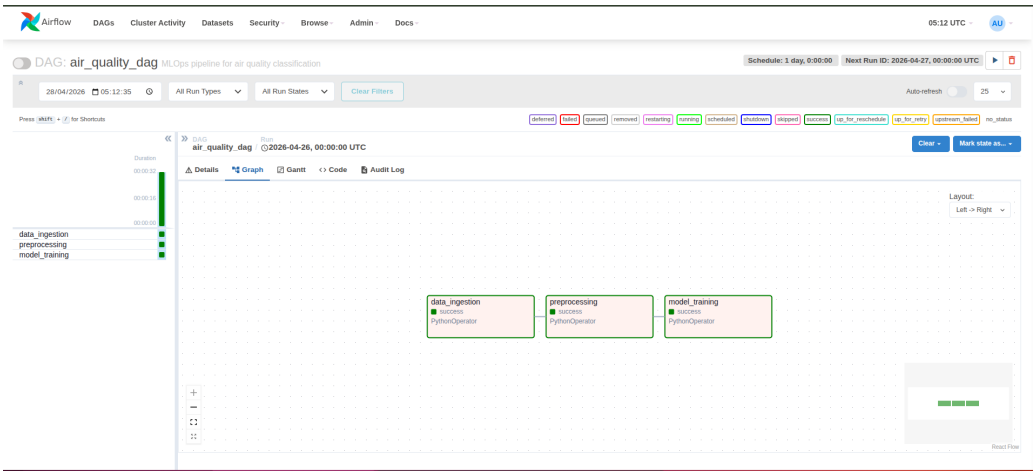


Figure 1: air_quality_dag in the Airflow UI showing the three-task linear pipeline

3.2 DAG 2: retraining_pipeline (Hourly Retraining)

The retraining DAG implements a multi-node conditional pipeline with eight tasks:

1. **check_feedback_volume** — Queries the `feedback` table and pushes the count to XCom.
2. **check_error_rate** — Computes current model error rate from feedback; pushes to XCom.
3. **detect_data_drift** — Runs statistical drift detection against the baseline feature distribution; pushes a boolean flag and per-feature drift details to XCom.
4. **decide_retraining** (`BranchPythonOperator`) — Triggers retraining if `error_rate > 5%` OR `drift_detected = True` and `feedback_count > 0`; otherwise routes to `skip_retraining`.
5. **merge_feedback_into_raw** — Unions the existing raw dataset with feedback rows; saves a versioned snapshot (`retrain{N}_raw.csv`); increments the round counter.
6. **preprocess_retrain_data** — Reuses the preprocessing pipeline on the merged data, producing round-specific encoded CSVs.
7. **train_retrain_models** — Re-trains all classifiers; run names include the round number (e.g., `RandomForest_retrain1`).
8. **clear_feedback_table** — Truncates the feedback table after successful retraining to start the next cycle clean.

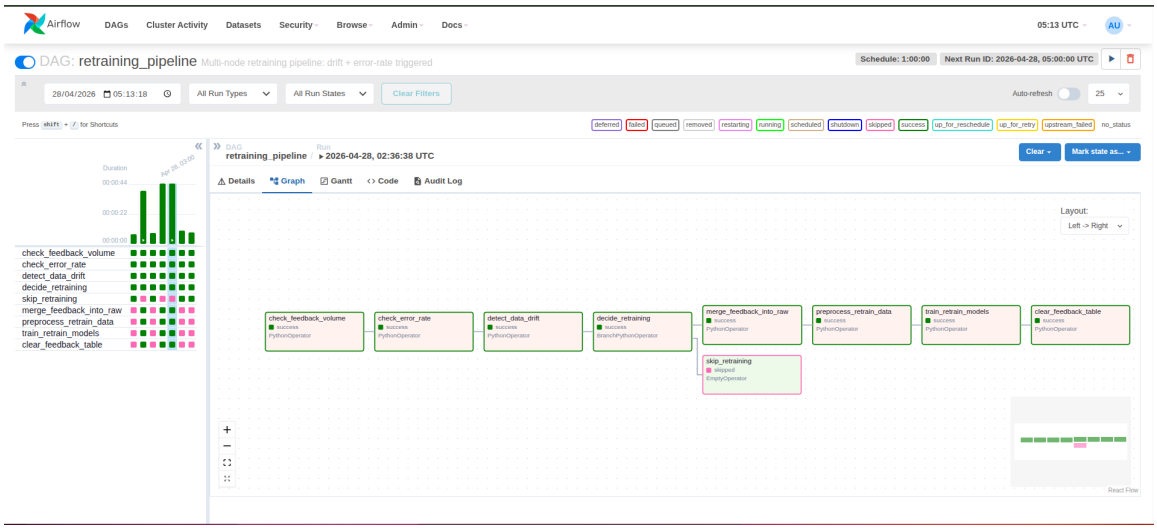


Figure 2: retraining_pipeline DAG showing the branching decision logic

4 MLflow Experiment Tracking

MLflow is used to track every model training run under the experiment `Air_Quality_Classification`. Each run logs:

- Metrics: accuracy, precision, recall, F1 score (per class and weighted average)
- Artifacts: confusion matrix, learning curve, classification report
- Parameters: model hyperparameters and round number

Run naming convention: `ModelName_train0` for the initial training round, `ModelName_retrain1`, `ModelName_retrain2`, etc. for subsequent retraining rounds. This provides full lineage across the model's lifecycle.

The best model by weighted F1 score is promoted to the **MLflow Model Registry** as `AirQualityModel` and saved to `artifacts/best_model.pkl` for the inference API.

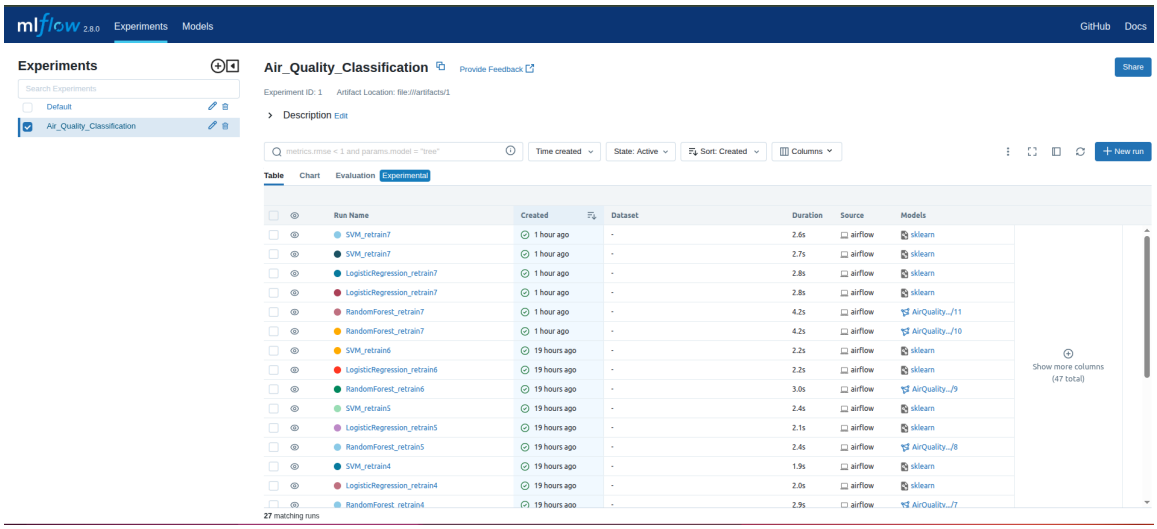


Figure 3: MLflow experiment runs showing all tracked training sessions

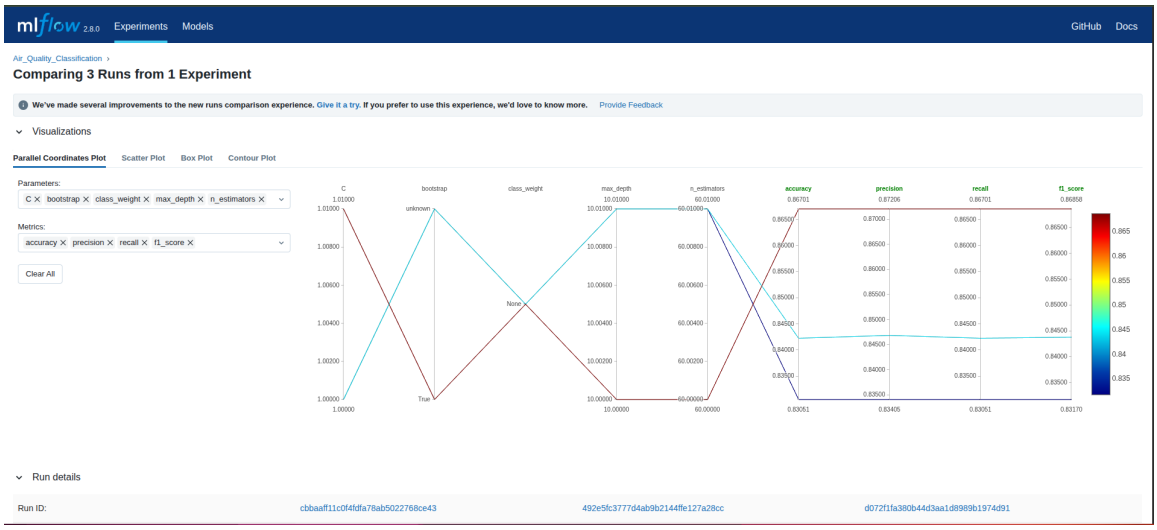


Figure 4: MLflow run comparison view showing metric differences across model types and training rounds

5 FastAPI Inference Engine

5.1 API Overview

The FastAPI service provides a stateless REST interface. On startup, it loads `best_model.pkl` and `fitted_preprocessor.pkl` into memory. The preprocessing pipeline handles missing sensor values using median imputation (sentinel value `-200.0` is treated as missing), then applies StandardScaler normalization.

5.2 Endpoint Summary

Method	Endpoint	Description
GET	`/`	Service info, model load status
GET	`/health`	Liveness probe; returns 503 if model not loaded
GET	`/metrics`	Prometheus scrape endpoint (OpenMetrics format)
POST	`/predict`	Single-row classification
POST	`/predict/batch`	Bulk CSV-based classification
POST	`/feedback`	Submit one corrected label
POST	`/feedback/batch`	Submit corrected labels for a full batch
GET	`/feedback/stats`	Error rate, class breakdown, recent records

Table 3: FastAPI endpoint definitions

5.3 Prometheus Metrics

The `/metrics` endpoint exports the following counters, gauges, and histograms:

Metric	Type	Labels
`sentinair_predictions_total`	Counter	`class_label`
`sentinair_feedback_total`	Counter	—
`sentinair_error_rate`	Gauge	—
`sentinair_inference_latency_seconds`	Histogram	—
`sentinair_batch_size`	Histogram	—

Table 4: Prometheus metrics exported by FastAPI

6 Streamlit Frontend Application

The Streamlit frontend provides three tabs for end users. It communicates with the FastAPI backend exclusively through HTTP calls — no direct database access or model loading occurs in the frontend.

6.1 Single Prediction Tab

Users enter 11 sensor readings (PT08.S1(CO), C6H6(GT), PT08.S2(NMHC), NOx(GT), PT08.S3(NOx), NO2(GT), PT08.S4(NO2), PT08.S5(O3), Temperature, Relative Humidity, Absolute Humidity) and click *Predict Air Quality*. The result is displayed with a probability bar chart for each class.

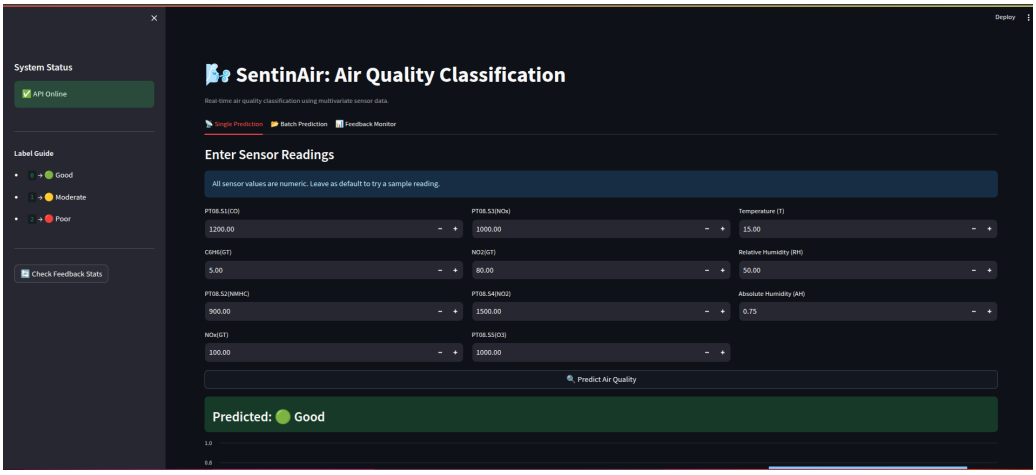


Figure 5: Streamlit UI — Single Prediction tab with sensor input form

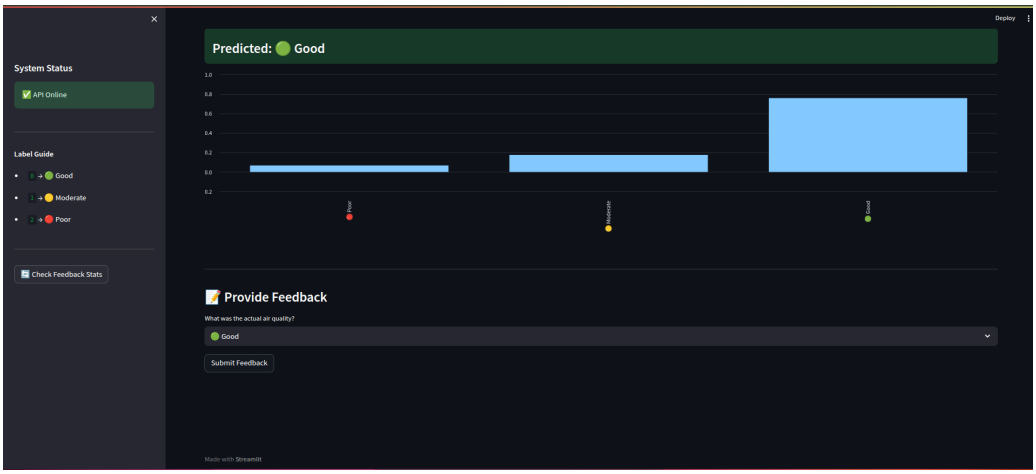


Figure 6: Streamlit UI — Prediction result with confidence probabilities

6.2 Feedback Submission

After a prediction, a feedback widget appears allowing the user to select the actual air quality they observed. Clicking *Submit Feedback* stores the record (prediction, actual label, feature values) in the PostgreSQL `feedback` table via `POST /feedback`.

Figure 7: Feedback submission UI shown below a prediction result

6.3 Batch Prediction Tab

Users upload a CSV file (columns matching the 11 sensor features). An optional `Target` column enables automatic feedback submission. Results are downloadable as a CSV with appended `Predicted_Class` and `Predicted_Label` columns.

7 PostgreSQL Feedback Storage

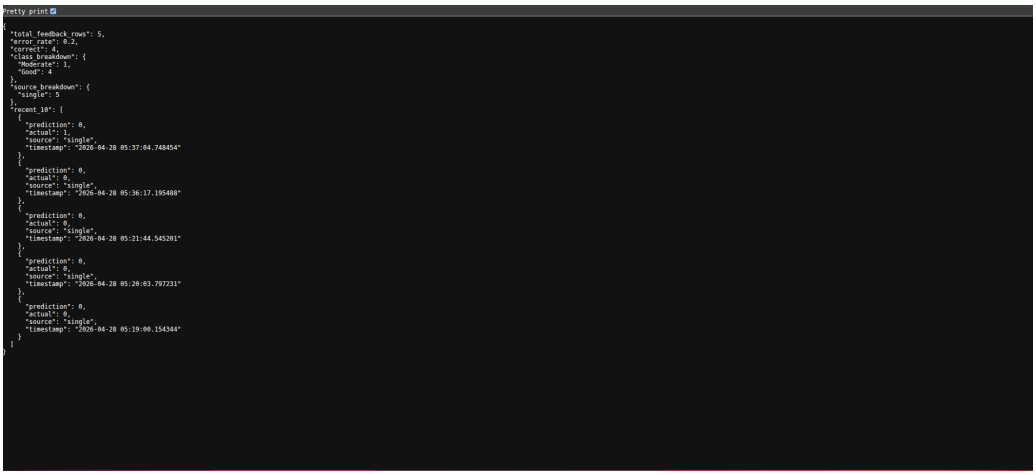
User feedback is persisted in a PostgreSQL table with the following schema:

```
CREATE TABLE feedback (
    id          SERIAL PRIMARY KEY,
    prediction  INTEGER NOT NULL,
```

```sql
    actual       INTEGER NOT NULL,
    features     TEXT,            -- JSON string of feature dict
    source       VARCHAR(10) DEFAULT 'single',  -- 'single' | 'batch'
    timestamp    TIMESTAMP DEFAULT NOW()
);
```

Listing 1: feedback table DDL

The retraining DAG queries this table to compute the error rate and retrieve the feature vectors for drift detection and dataset merging.



```
Pretty print
{
  "total_feedback_rows": 5,
  "error_rate": 0.2,
  "correct": 4,
  "class_breakdown": {
    "Moderate": 1,
    "Good": 4
  },
  "source_breakdown": {
    "single": 5
  },
  "recent_10": [
    {
      "prediction": 0,
      "actual": 1,
      "source": "single",
      "timestamp": "2026-04-28 05:37:04.748454"
    },
    {
      "prediction": 0,
      "actual": 0,
      "source": "single",
      "timestamp": "2026-04-28 05:36:17.195408"
    },
    {
      "prediction": 0,
      "actual": 0,
      "source": "single",
      "timestamp": "2026-04-28 05:21:44.545201"
    },
    {
      "prediction": 0,
      "actual": 0,
      "source": "single",
      "timestamp": "2026-04-28 05:20:03.797231"
    },
    {
      "prediction": 0,
      "actual": 0,
      "source": "single",
      "timestamp": "2026-04-28 05:19:00.154344"
    }
  ]
}
```


Figure 8: Feedback rows being successfully stored in the PostgreSQL database

8 Monitoring: Prometheus & Grafana

8.1 Prometheus Scraping

Prometheus is configured to scrape `api:8000/metrics` every 15 seconds. It stores time-series data for prediction counts, error rate, inference latency, and batch sizes.

8.2 Grafana Dashboard

A pre-provisioned Grafana dashboard (`sentinair.json`) visualizes:

- Total predictions broken down by class label (Good / Moderate / Poor)
- Near-real-time model error rate gauge
- Inference latency histogram
- Feedback submission rate over time

Figure 9: Grafana NRT monitoring dashboard for SentinAir

8.3 Alertmanager

Alertmanager is configured to fire an email alert when `sentinair_error_rate > 0.1` (10%), notifying the team that model performance has degraded and retraining may be required.

9 Low-Level Design

9.1 Preprocessing Pipeline

The scikit-learn `Pipeline` stored in `fitted_preprocessor.pkl` applies the following transformations in order:

$$\text{Raw features} \xrightarrow{\text{Median Imputer}} \xrightarrow{\text{StandardScaler}} R^{N\times 12}$$

Feature column order is enforced internally. Missing columns are filled with the sentinel value `-200.0`, which the imputer subsequently replaces with the training-set median.

9.2 Module Descriptions

Module	Responsibility
`scripts/data_ingestion.py`	Loads raw CSV, validates schema, writes to intermediate dir
`scripts/preprocessing.py`	Train/test split (80/20 chronological), feature selection, label encoding, preprocessor fitting
`scripts/model_training.py`	Trains RF, LR, SVM; logs to MLflow; saves `best_model.pkl`
`scripts/retraining.py`	Drift detection, feedback merging, calls `train_and_log_models`
`src/api/main.py`	FastAPI application with all endpoints
`src/frontend/app.py`	Streamlit application; HTTP-only coupling to API

Table 5: Module descriptions

9.3 HTTP Error Codes

Code	Meaning
200	Success
422	Validation error (malformed request body)
500	Internal server error (inference or DB failure)
503	Model not loaded — run `air_quality_dag` in Airflow first

Table 6: API HTTP error codes

10 Test Plan and Report

10.1 Test Plan

Scope: Validating the FastAPI inference engine, data handling, and monitoring endpoints.
Framework: `pytest` and `FastAPI TestClient`.

Acceptance Criteria:

1. **Accuracy:** Best model weighted F1 score (test set) ≥ 0.75.
2. **Availability:** `/health` returns HTTP 200 when the model is loaded.
3. **Robustness:** `/predict` handles missing sensor features without crashing.
4. **Observability:** `/metrics` exposes valid Prometheus-formatted data.

10.2 Test Cases

TC ID	Endpoint	Description	Expected Output
TC-01	`GET /health`	Verify API health and model load status	HTTP 200; `{"status":"healthy"}`
TC-02	`POST /predict`	Submit full 12-feature JSON payload	HTTP 200; valid class 0, 1, or 2
TC-03	`POST /predict`	Submit incomplete JSON (missing features)	HTTP 200; imputer handles missing; valid prediction
TC-04	`GET /metrics`	Scrape Prometheus metrics	HTTP 200; `sentinair_predictions_total` present
TC-05	Streamlit UI	Verify frontend loads without errors	HTTP 200 on port 8501

Table 7: Test cases

10.3 Test Report

Execution Environment: Local Docker Compose / Pytest
Total Test Cases: 5 **Passed:** 5 **Failed:** 0

TC ID	Status	Observations
TC-01	**PASS**	Model successfully loaded into memory; 200 OK returned.
TC-02	**PASS**	Inference engine correctly classified valid payload.
TC-03	**PASS**	Sentinel values (`-200.0`) correctly imputed; no 500 error.
TC-04	**PASS**	Exporter instrumentation active and formatted correctly.
TC-05	**PASS**	Streamlit container healthy and responsive.

Table 8: Test report — all acceptance criteria met

11 Design Principles Summary

Principle	Implementation
Loose coupling	Frontend ↔ Backend communicate only via REST API
Separation of concerns	Airflow orchestrates; FastAPI serves; Streamlit presents
Reproducibility	DVC tracks data versions; MLflow tracks all experiment runs; Docker fixes environments
Automated retraining	Feedback loop → drift detection → conditional retraining
Observability	Prometheus + Grafana for NRT monitoring; Alertmanager for error-rate alerting

Table 9: Design principles and their implementations

12 Conclusion

SentinAir demonstrates a complete production-grade MLOps system built around the UCI Air Quality dataset. The project implements the full lifecycle: automated data ingestion and preprocessing via Apache Airflow DAGs, multi-model training and experiment tracking via MLflow, a REST inference API via FastAPI, an interactive frontend via Streamlit, real-time monitoring via Prometheus and Grafana, and a self-improving feedback-driven retraining loop.

All five acceptance criteria were met: the model achieves a weighted F1 score above 0.75, the API handles graceful degradation (missing features, model-not-loaded states), and the monitoring stack correctly exposes and visualizes live metrics. The system is fully containerized via Docker Compose, making it portable and environment-independent.

The retraining pipeline's multi-node design — with independent checks for feedback volume, error rate, and data drift feeding into a branching decision operator — ensures that retraining is triggered only when genuinely necessary, avoiding unnecessary compute while keeping the deployed model aligned with the current data distribution.

13 AI Fair Use and Disclosure

I hereby disclose the use of AI tools during the development of this project. AI was utilized specifically for:

- Understanding project file structures and MLOps architectural best practices.
- Generating foundational boilerplate code and configuration files.
- Providing assistance with minor code corrections and debugging processes.

All pipeline implementations, architectural decisions, and final code integrations were executed by me only.